



ESTRUTURA DE DADOS I

FUNDAMENTOS E ESTRUTURAS LINEARES

Prof Me Wesley Soares de Souza

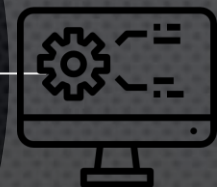
✦
👤 Boas-vindas

MEMÓRIA E ALOCAÇÃO DINÂMICA

Como os dados realmente ocupam espaço durante a execução de um programa?

```
int idade = 30;
```

ONDE O VALOR **30** ESTÁ ARMAZENADO?



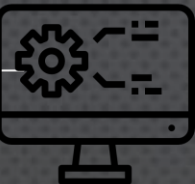
POR QUE ESTUDAR MEMÓRIA?

EM ESTRUTURA DE DADOS, NÃO BASTA SABER:

- **“COMO ARMAZENAR UM DADO?”**

TAMBÉM PRECISAMOS ENTENDER:

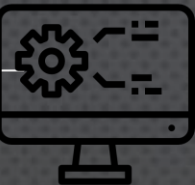
- **“COMO ESSE DADO É ARMAZENADO NA MEMÓRIA?”**



POR QUE ESTUDAR MEMÓRIA?

○ QUE INFLUENCIA:

- DESEMPENHO;
- CONSUMO DE MEMÓRIA;
- ACESSO AOS DADOS;
- CRIAÇÃO DE ESTRUTURAS;
- INSERÇÃO E REMOÇÃO;
- ESCALABILIDADE.

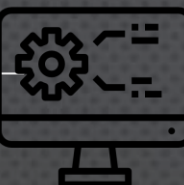
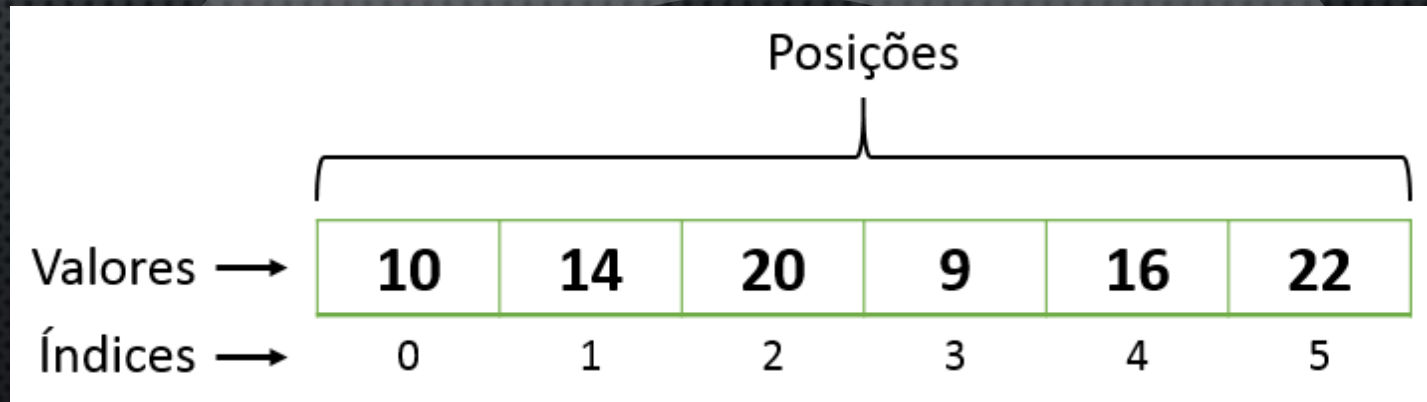


MEMÓRIA RAM

A MEMÓRIA RAM PODE SER ENTENDIDA, DE FORMA SIMPLIFICADA, COMO UMA GRANDE SEQUÊNCIA DE POSIÇÕES CAPAZES DE ARMAZENAR INFORMAÇÕES.

CADA POSIÇÃO POSSUI:

- UM ENDEREÇO;
- UM CONTEÚDO.

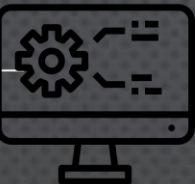


ENDEREÇO E CONTEÚDO

`int idade = 30;`



Memória	
0X1000	30

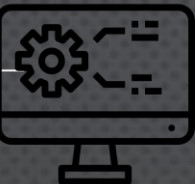


VARIÁVEIS EM JAVA

```
int idade = 30;  
double salario = 5000.0;  
boolean ativo = true;
```

- Identifique a variável, o tipo e o valor

Essas variáveis armazenam diretamente seus valores?



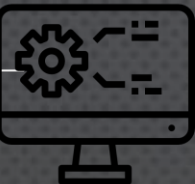
TIPOS PRIMITIVOS X REFERÊNCIAS

```
int idade = 30;  
double salario = 5000.0;  
boolean ativo = true;
```

Tipos primitivos

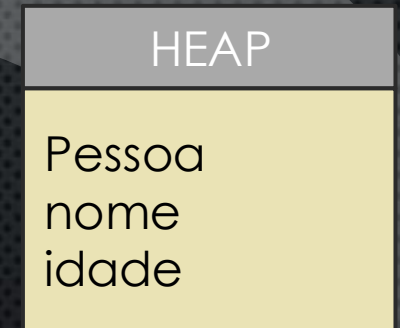
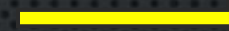
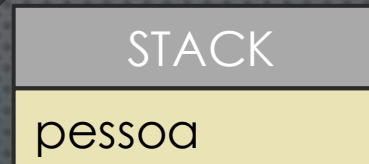
```
Pessoa pessoa;  
String nome;  
int[] numeros;
```

Tipos por referência



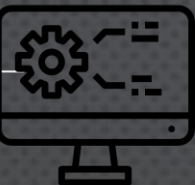
O QUE É UMA REFERÊNCIA?

```
Pessoa pessoa = new Pessoa();
```



```
new Pessoa();
```

Foi criado um objeto que pode ser acessado



STACK E HEAP

STACK

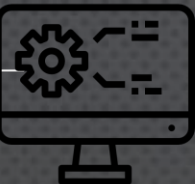
Usada, de forma simplificada, para:

- chamadas de métodos;
- variáveis locais;
- informações relacionadas à execução dos métodos.

HEAP

Usada para armazenar: objetos;

- arrays;
- estruturas criadas dinamicamente.



○ OPERADOR **NEW**

new Pessoa()

criação do objeto

Heap

referência retornada

pessoa



STACK

pessoa
idade = 30



HEAP

Pessoa
nome = "Wesley"
idade = 30

DUAS REFERÊNCIAS PARA O MESMO OBJETO

```
Pessoa p1 = new Pessoa();  
Pessoa p2 = p1;
```



STACK

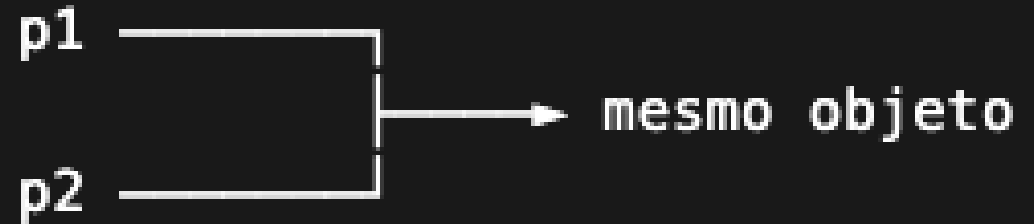


HEAP

```
Pessoa  
nome = null  
idade = 0
```


ALTERANDO ATRAVÉS DE UMA REFERÊNCIA

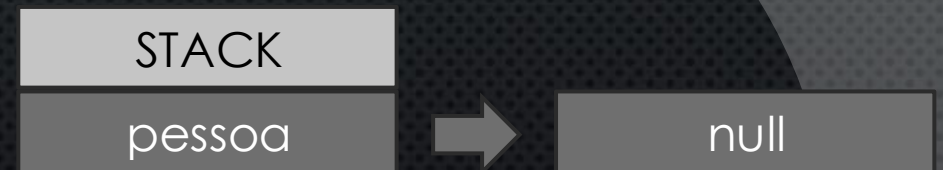
```
Pessoa p1 = new Pessoa();  
Pessoa p2 = p1;  
p2.setNome("Carlos");
```



Se eu mostrar `p1.getNome()` o que irá aparecer?

NULL

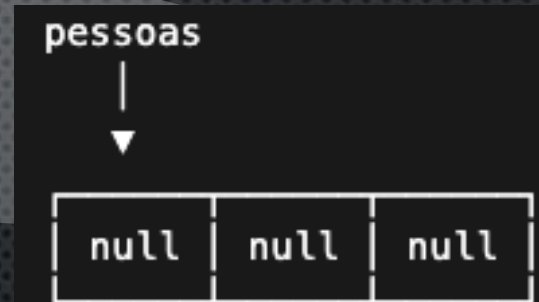
- `PESSOA PESSOA = NULL;`
 - NÃO EXISTE OBJETO ASSOCIADO À REFERÊNCIA.
- `PESSOA.GETNOME();`
 - `NullPointerException`



CRIANDO OBJETOS DINAMICAMENTE

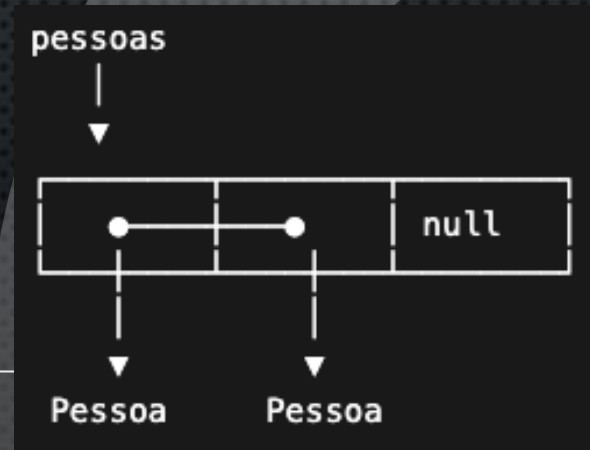
```
Pessoa[] pessoas = new Pessoa[3];
```

- TEMOS UM ARRAY CRIADO NO HEAP PORÉM SEM VALORES (NULL)



- PORÉM:

- PESSOAS[0] = NEW PESSOA();
- PESSOAS[1] = NEW PESSOA();



O QUE SIGNIFICA "ALOCAÇÃO DINÂMICA"?



Criar estruturas durante a execução do programa conforme a necessidade.

```
Pessoa[] pessoas = new Pessoa[3];
```

- Declara uma **variável de referência** chamada pessoa.
- **new Pessoa()**: Cria um novo objeto Pessoa durante a execução do programa.

STACK

pessoa

HEAP

Pessoa

nome

idade

GARBAGE COLLECTOR

O Garbage Collector (GC) identifica objetos que não podem mais ser alcançados pelo programa e pode liberar a memória associada a eles.



- Diferente da linguagem C que é necessário limpar a memória manualmente (malloc e free)
 - `Pessoa p = new Pessoa();`
 - `p = null;`
 - `p = null`: não significa que o objeto é imediatamente apagado da memória, porém esta na mira do garbage collector.
-

COMO O GARBAGE COLLECTOR PENSA?

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Pessoa p1 = new Pessoa();  
        Pessoa p2 = new Pessoa();  
  
        p1.setNome("Ana");  
        p2.setNome("Carlos");  
  
        p1 = null;  
  
    }  
}
```

Perguntas

Quantos objetos **Pessoa** foram criados?

Quantas referências existem inicialmente?

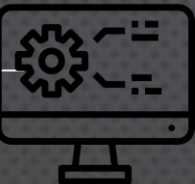
O que acontece com o primeiro objeto depois de **p1 = null**?

O segundo objeto ainda pode ser acessado?

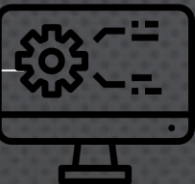
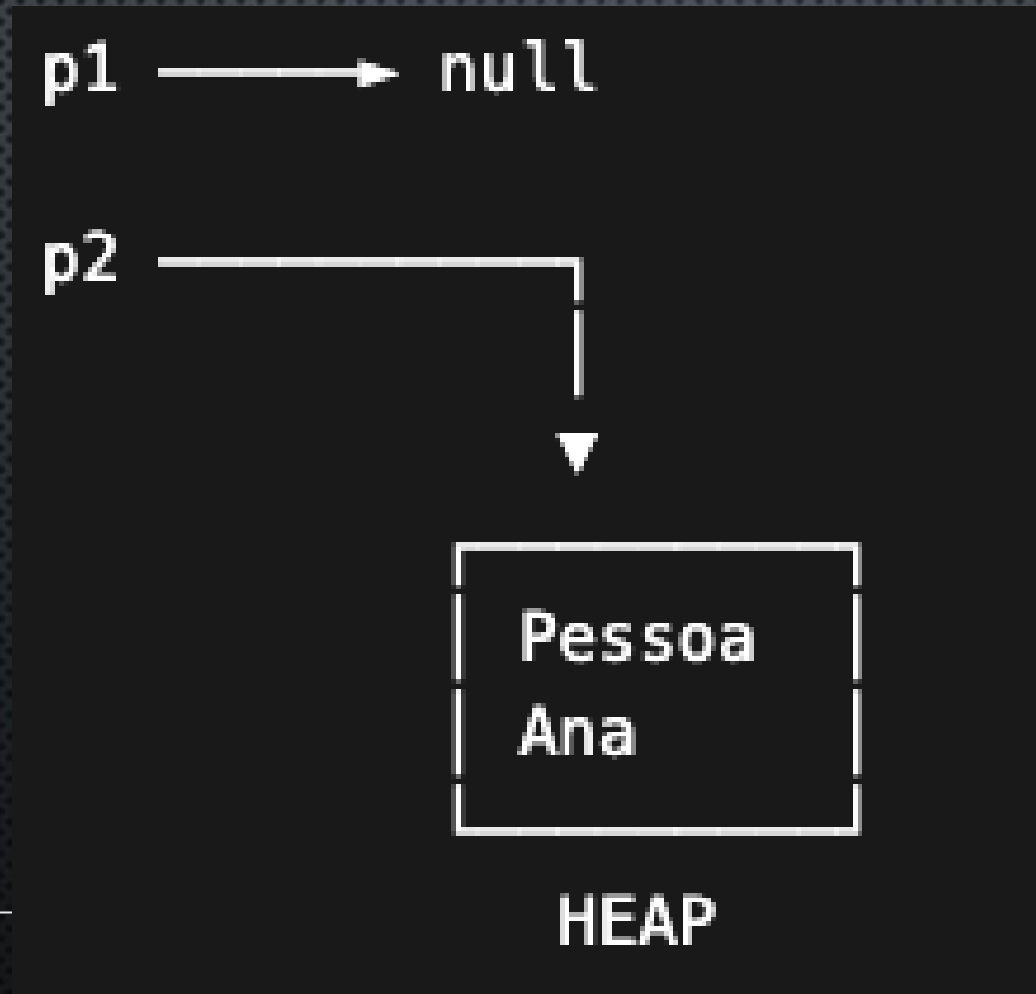


ATIVIDADE: DESENHANDO STACK E HEAP

```
Pessoa p1 = new Pessoa();  
Pessoa p2 = new Pessoa();  
  
p1.setNome("Ana");  
  
p2 = p1;  
  
p1 = null;
```



ATIVIDADE: DESENHANDO STACK E HEAP



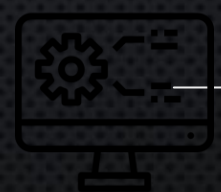
POR QUE ISSO É IMPORTANTE EM ESTRUTURA DE DADOS?

1

```
class No {  
    int valor;  
    No proximo;  
}
```

2

```
No primeiro = new No();  
No segundo = new No();  
primeiro.proximo = segundo;
```



POR QUE ISSO É IMPORTANTE EM ESTRUTURA DE DADOS?

primeiro



```
valor = 10  
proximo
```



```
valor = 20  
proximo  
null
```

Lista ligada

DA MEMÓRIA PARA AS ESTRUTURAS DE DADOS

Memória

Variáveis

Referências

Objetos

Nós

Listas ligadas



Pilhas dinâmicas

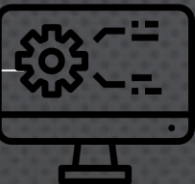
Filas dinâmicas

Árvores



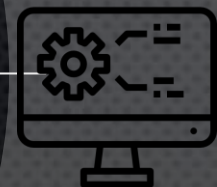
EXERCÍCIO

- Crie uma classe chamada No em java com valor (int) e próximo (No).
- Deverá ter um construtor inicializando valor
- Crie uma lista ligada com Nó 1, 2 e 3.



EXERCÍCIO

1. QUANTOS OBJETOS FORAM CRIADOS?
2. QUANTAS REFERÊNCIAS NO EXISTEM?
3. QUAL É O VALOR DE: `N1.PROXIMO.VALOR`
4. QUAL É O VALOR DE: `N1.PROXIMO.PROXIMO.VALOR`
5. O QUE ACONTECE SE FIZERMOS: `N2 = NULL;`



DESAFIO FINAL

No N1 = NEW No(10);

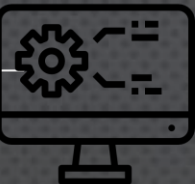
No N2 = NEW No(20);

No N3 = NEW No(30);

N1.PROXIMO = N2;

N2.PROXIMO = N3;

N2 = NULL;



ATÉ A PRÓXIMA!

Continue estudando,
praticando e nunca pare
de evoluir!

NOSSA JORNADA
ESTÁ SÓ COMEÇANDO!

VALEU!

ATÉ
MAIS!

